

Same inputs, same factory: why determinism matters in 2026

This is the paper in ASD-STE100 Simplified Technical English, after two passes (log). WHITEPAPER.pdf prints this text. The original paper, in colour, is PICASSOPAPER.pdf. The source, the render scripts and the rebuild experiment are in docs/whitepapers/determinism-2026.

A factory that improves itself from run to run does an experiment. An experiment gives a result only when all items, other than the treatment, are fixed. This paper examines the evidence that reproducibility is a requirement in 2026. It shows that functional package management gives reproducibility at scale. It reads the ouroboric-dev repositories to find what they pin and what they leave to chance. It proposes six specific ways for the Nix labs of picasso to close the gap.

Matheus Hoffmann · Escola Politécnica, Universidade Federal do Rio de Janeiro · ORCID 0009-0009-1056-7661 · version 1.0, 12 October 2026

Cite as: Hoffmann, M. (2026). *Same inputs, same factory: why determinism matters in 2026* (Version 1.0). Zenodo. https://doi.org/10.5281/zenodo.XXXXXXXX
Drafted with the assistance of Claude Code (Anthropic) and reviewed by the author.



Abstract

Background. In 2026, three pressures occur at the same time. Computational results frequently do not run again: of 9,000+ R files in published replication packages, 74% failed on their first execution [17]. Of 1.4 million Jupyter notebooks, 4.03% gave their own results again [18]. Supply-chain attacks aim at the gap between source and binary [8, 9, 10]. Under the EU Cyber Resilience Act, the obligations to report started to apply on 11 September 2026 [12]. Coding agents put a stochastic generator in the build loop: at temperature 0, 1,000 identical requests to a production LLM endpoint gave 80 different completions [24].

Argument. If the generator is stochastic, the remainder of the system must be deterministic. Otherwise, it is not possible to relate a measured change to its cause. Functional package management makes the environment a pure function of declared inputs. A 2025 study rebuilt 709,816 nixpkgs packages. It found that 69–91% were bitwise reproducible and that more than 99% were rebuildable [5].

Case. ouroboric-dev already pins its factory version, its skills, its config hash and its agent CLI versions. It states that "benchmarks are only comparable with the same skills" [29]. Its specs give

less information about how it pins the operating-system layer, the host tools and the container images. We show that the pinned nixpkgs of picasso contains packages for 12 of the 14 tools that its check ladder and sandbox name. We also show that an OCI image from these tools rebuilds bit-for-bit (§5).

Proposal. We propose six additions, from a `labs/factory` lab to an environment hash in the Yggdrasil ledger. We map each addition to a section of the Yggdrasil v0 spec. We also give an evaluation plan, and the limits of what Nix can fix and cannot fix.

1. Introduction

Ouroboric (ouroboric.dev) builds "open factories that improve themselves". Its first factories are Yggdrasil, a software factory, and Asgard, a company factory that runs on Yggdrasil [29]. Its charter makes measurement the core claim: "every claim about a factory rests on runs". The first north-star measure is that "each factory measurably improves per iteration on its own benchmarks: hidden acceptance score, first-pass rate and cost per task, run against run."

That is a scientific claim, so it has the conditions of a scientific claim. Assume that run *B* is better than run *A* because of a change to a prompt, a graph or a model. To make this claim, it is necessary to know that no other item changed, or to measure the items that changed. Some changes can move each metric that is important to the charter, with no change to the factory. Examples are a compiler that updated overnight, a base image whose `apt-get install` resolved differently, or a linter whose new release flags new findings.

picasso (github.com/h0ffmann/picasso, formerly `nix-config`) is a set of self-contained Nix flakes, called labs. Other repositories use a lab as a flake input, a sparse submodule or a GitHub Action. One line gives its main rule: "a toolchain is a dependency, so pin it like one" [30]. This paper asks what that rule is worth to ouroboric-dev. It answers with evidence from the literature, from the ouroboric-dev repositories and from two rebuild experiments.

Method. We read all Markdown files in ouroboric-dev/ouroboric (4ab80ed), yggdrasil (54ea51f), asgard (8bdbc03) and .github (b799793), at their heads on 12 October 2026. We found the references with a literature search. Before we cited a reference, we checked it against a primary record (Crossref, arXiv, the publisher or the official register). Each number that we quote comes from the checked record. We wrote no reference from memory.

2. Definitions

In practice, people use these terms loosely. This section gives a fixed meaning for each term.

term	meaning in this paper	source
reproducibility	to get consistent results with the same input data, computational steps, methods, code and conditions of analysis	NASEM [15]
replicability	to get consistent results across studies about the same question, each study with its own data	NASEM [15]
reproducible build	"given the same source code, build environment and build instructions, any party can recreate bit-by-bit identical copies of all specified artifacts"	reproducible-builds.org [28]
rebuildability	a build that still succeeds from its recorded inputs, if the output is bit-identical or not	Malka et al. [5]
deterministic component	a component whose output is a function of its declared inputs only; here: the environment, the tools, the checks and the grader	this paper
stochastic component	a component whose output changes for fixed declared inputs; here: model inference behind an agent CLI	this paper, after [23, 24]

The difference between the last two rows is the central tool of this paper. We do not claim that it is possible to make agents deterministic. We claim that it is possible to make all items around the agents deterministic. This makes it possible to measure the variance of the agents.

3. Why determinism, why 2026

3.1 Computational results do not re-run by default

The reproducibility deficit is a measured fact, not an anecdote. A *Nature* survey asked 1,576 researchers. More than 70% tried to reproduce the experiments of another scientist and failed. More than half failed to reproduce their own experiments [13].

When researchers publish code, frequently the code does not run. Trisovic et al. ran more than 9,000 R files from 2,000+ Harvard Dataverse replication datasets in a clean environment. Of these files, 74% did not complete without an error. After an automatic tool cleaned the code, 56% did not complete without an error [17].

Pimentel et al. ran 1.4 million Jupyter notebooks from GitHub again. Of these, 24.11% ran without errors, and 4.03% gave the same results [18]. In AI research, a survey of 400 IJCAI and AAAI papers found that no paper documented all the variables necessary to reproduce its results. For each factor, the papers documented 20–30% of the variables [19]. Collberg and Proebsting tried to build the artefacts of published papers themselves. They got the same conclusion for computer-systems research [16].

Peng argues that computational science needs the code and the data together with the paper [14]. The FAIR principles demand that research objects are reusable [21]. These two conditions are necessary, but these studies show that they are not sufficient: code without its environment decays. Hinsen calls this *software collapse*: software does not work any more because the layers below it change, not because the software changed [20]. Researchers proposed containers as the remedy [22], and a container freezes a result. But a Dockerfile that runs `apt-get update` describes a different image on each day that it is built.

3.2 The supply chain attacks the space between source and binary

Thompson showed in 1984 that a compromised compiler can insert a backdoor that appears in no source code [7]. Forty years later, the attack surface is the full dependency graph. Ohm et al. collected 174 malicious packages that attackers used in real attacks [8]. The taxonomy of Ladisa et al. for attacks on open-source supply chains lists 107 attack vectors [9].

The xz-utils backdoor (CVE-2024-3094) is the case study for this paper. The malicious code was in the *upstream tarballs* of xz 5.6.0 and later versions. Also, "the liblzma build process extracts a prebuilt object file from a disguised test file" [10].

The reproducible-builds community argued for this defence: independent parties derive a build again from audited inputs, and they get the same bits [6]. If two builders get different results, an item in the chain is not what it claims to be. SLSA, the industry framework for build integrity, grades build platforms by provenance and isolation [11]. Reproducibility lets a third party check that provenance.

3.3 Regulation now asks for it

The EU Cyber Resilience Act, Regulation (EU) 2024/2847, applies to manufacturers of products with digital elements. It requires them to identify and document the components of their products, "including by drawing up a software bill of materials". The software bill of materials must cover at least the top-level dependencies (Annex I, Part II) [12]. Its obligations to report vulnerabilities and incidents started to apply on 11 September 2026. Most other obligations apply from 11 December 2027 [12].

An SBOM is a claim about what is in an artefact. A build in which each input is a hashed store path makes that claim checkable, not only declarative. Ouroboric does not ship products into the EU market today. But the products that its factories build can enter that market. The charter says that what operators build "is yours" [29]. Thus, the output of the factory should come with evidence that an operator can use.

3.4 Agents put a stochastic generator inside the loop

The new element in 2026 is the generator. Ouyang et al. sent the same 829 code-generation problems to ChatGPT many times. In some tasks, the requests gave no equal test outputs. The share of these tasks was 75.76% on CodeContests, 51.00% on APPS and 47.56% on HumanEval. Temperature 0 decreased the variation, but did not remove it [23].

He (Thinking Machines Lab, 2025) found the cause of the residual nondeterminism of inference servers: kernels that are not batch-invariant. Server load changes the batch size, so the output for one user depends on the traffic of other users. At temperature 0, 1,000 identical requests to Qwen3-235B gave 80 different completions. With batch-invariant kernels, all 1,000 completions were identical [24].

Benchmarks are fragile too. Yggdrasil scores its hidden suite "SWE-bench style" (spec §11), after Jimenez et al. [26]. Zhu et al. show that flaws in the task configuration or the reward design can make the estimate of agent performance incorrect. The error can be up to 100% in relative terms, too high or too low [27].

This has two consequences for a factory. The first consequence is about **variance budgets**. If the model adds irreducible variance, it is necessary to remove all other sources of variance. If not, it is not

possible to distinguish a measured improvement from noise.

The second consequence is about the **attack surface**. Prompt injection lets text in a repository steer an agent [25]. Thus, the environment of an agent is a part of its security boundary. An environment that changes between runs is a boundary that nobody reviewed.

> **The thesis.** In 2026, the question is no longer whether to make a build reproducible. The question is where to put the line between the fixed items and the measured items. A factory that improves itself puts this line around the model. The environment, the tools, the checks and the grader are deterministic and hashed. The model is the only stochastic input. The factory estimates its variance with repeated runs, and does not assume that there is no variance.

4. The mechanism: functional package management

Dolstra and de Jonge introduced Nix in 2004 [1]. Nix treats a build as a pure function. Nix keeps each package under a path that is a cryptographic hash of all its inputs (sources, dependencies, build script, compiler flags). Builds run in a sandbox without network access. The only exception is fixed-output fetches, which declare their content hash before the fetch. Two packages that differ in one or more inputs cannot collide.

NixOS extended this model to a full operating system [2]. GNU Guix applied the same model to HPC [3]. Flakes add a lock file that pins each input to a revision and a content hash. The labs of picasso commit this lock file.

Recent studies tested the model at the scale of the largest software distribution. Malka, Zacchiroli and Zimmermann used historical data to reconstruct the build environments of about 7 million Nix packages. They rebuilt 99.94% of the 14,000 packages of a nixpkgs revision that was six years old [4].

Their next study rebuilt 709,816 packages from nixpkgs snapshots, with samples from 2017 to 2023. Of these packages, 69–91% were bitwise reproducible, and this share increased over time. Rebuildability was more than 99%. About 15% of the failures came from embedded build dates [5].

property	Dockerfile + package manager	devcontainer.json	Nix flake (picasso lab)
what is pinned	base image tag (a pointer that moves if no digest is used); packages are resolved at build time	an image or Dockerfile, plus features	each input, transitively, by revision and content hash in `flake.lock`
same file, later build	a different image each time a mirror or registry moved	same as its Dockerfile	the same store paths; the outputs are bit-identical for the 69–91% reproducible share [5]
network during build	yes	yes	only fixed-output fetches with a declared hash
identity of the environment	an image digest, known only after the build	same	the hash of the lock file, known before the build
upgrade	edit and rebuild; the old state is lost if it has no tag	same	a one-file diff to `flake.lock`, reviewable and revertible
independent verification	rarely possible	rarely possible	`nix build --check` rebuilds and compares bits

Table 1. The properties are as each tool specifies them. The reproducibility figures are measured values [4, 5]. Docker images that Nix builds (`dockerTools`) combine the right column with the container runtime that Yggdrasil already selected.

5. Case study: ouroboric-dev on 12 October 2026

5.1 What it already holds fixed

The documents of ouroboric-dev show that the project gives unusual attention to determinism at the level of the factory. Table 2 gives the practices that we found.

practice	evidence	what it controls
the factory never builds itself in place	"the running factory MUST always be the last released version pinned in L0" (yggdrasil spec §15.4)	the factory version
vendored, locked skills	`skills.lock` with a hash for each file; "benchmarks are only comparable with the same skills" (ouroboric writing-styles §5)	agent instructions
agent CLIs pinned	"agent CLIs pinned to exact versions with auto-update off" (spec §6.1)	the agent harness
run provenance	each report records the factory version and the config hash; `ygg metrics --compare` prints the differences (spec §10.2)	traceability of numbers
tamper-evident ledger	each JSONL line contains the hash of the line before it (spec §10.1)	the record of a run
deterministic end-to-end tests	the `scripted` adapter runs full graphs at zero model cost; "this is what CI runs" (spec §13)	the logic of the factory, without the model
isolated grader	a hidden suite in a separate container, which starts after the agent containers stop (spec §9.5, §11)	benchmark leakage
pinned diagram toolchain	Kroki is pinned to 0.32.1; a check compares Inter against a pinned SHA-256 (ouroboric diagrams.md §4)	rendered docs
deterministic routing	routing bundles come from "deterministic exec nodes … with no model calls" (spec §8.1)	context assembly

Table 2. Each spec reference is to docs/2-engineering/specs/2026-10-11-yggdrasil-v0-spec.md in yggdrasil at 54ea51f.

5.2 What it leaves to chance

The same documents give less information about the layer below the factory. The list below gives the gaps that we infer from the text. No gap is a defect in a v0 design. Several gaps are already open items.

1. **Images and system packages.** The stack layer of the web realm is a `Dockerfile`, and the grader environment is also a `Dockerfile` (spec §9.4, §11). The factory layer adds an egress firewall, managed settings and a non-root user (§6.1). We searched all four repositories for *digest*, *sha256* and *apt-get*. The search found no rule that pins base images or system packages. The grader is the instrument that measures the hidden acceptance score, which is the first north-star measure. If each run rebuilds the instrument from a base that moves, the instrument is not the same from one run to the next.
2. **The host toolchain.** The spec says: "The host needs only Podman, the devcontainer CLI and the `ygg` binary" (§6.1). The check ladder also needs gofmt, golangci-lint, go vet, eslint, tsc, Vale, gitleaks, govulncheck, npm audit, oasdiff and Playwright (§9.5). The ledger does not record the versions of these tools. The `agent` field of the ledger contains the adapter, the model and the CLI version (§10.1). A new release of golangci-lint can change the first-pass rate.
3. **Machine facts that changed the design.** The first publish found three facts. Podman ran through a rootful socket. Podman could not mount WSL paths. A work credential helper ran on public image pulls. The journal says: "three of the four surprises here were facts about the machine, not about the design" (journal, 2026-10-11 first publish; spec §16 items 5–7).
4. **Pins versus patches.** The web-stack research states the conflict directly: "reproducibility (same versions on every run) and safety (latest patch) pull in opposite directions" (research-web-stack §5.4). The research resolves the conflict with a pinned minor line, a mandatory patch floor and a canary rollout (research-web-stack §5.4, spec §9.6).

5.3 Two rebuild experiments

We wanted to know if the right column of Table 1 is true for the tools of ouroboric itself. Thus we evaluated the nixpkgs pin of the picasso lint lab (`eaad0894`, 11 September 2026). We used Nix 2.34.6 in the container of this session.

tool that the yggdrasil spec names	in nixpkgs at eaad0894	tool	in nixpkgs at eaad0894
go	1.26.7	osv-scanner	2.5.1
golangci-lint	2.13.2	podman	5.8.6
govulncheck	1.8.0	devcontainer (CLI)	0.89.0
gitleaks	8.30.1	d2	0.8.1
vale	3.17.1	zensical	0.0.60
claude-code	2.1.266	opencode	1.18.30
oasdiff	**absent**	kroki	**absent** (it runs as an image)

Table 3. The output of `nix eval` on `<pkg>.version` for each attribute. 12 of 14 tools are present.

Experiment A. We fetched GNU hello 2.12.3 from cache.nixos.org. Then we rebuilt it locally with `nix-build -A hello --check`. This command rebuilds the derivation and fails if one bit of the output is different. The check passed. The binary from the NixOS build farm and the binary from an unrelated cloud container were identical.

Experiment B. We built an OCI image with `dockerTools.buildLayeredImage`. The image contains bash, coreutils, git, go, gitleaks, govulncheck, golangci-lint, Vale and osv-scanner. The expression is `image.nix`, next to the source of this paper. The first build took 42 s and made a 285 MB image with SHA-256 `6763d944…0b9a38` at store path `ls6nda0b…-factory-layer-demo.tar.gz`. `nix-build --check` rebuilt it in 21 s and found that it was bit-identical. The lock alone gave the identity of the image before the build.

> **What the experiments show, and what they do not.** The experiments show that a factory-layer image of the check tools of ouroboric itself is reproducible to the bit. This is true for a second build from the same pin. They do not show that this image is reproducible across machines. We had one machine, and Experiment A covers two builders for one package. Also, the experiments say nothing about model inference, which no package manager controls.

6. What picasso can do for ouroboric-dev



Each proposal obeys the lab contract in the AGENTS.md of picasso. Each lab is one self-contained flake and refers to nothing outside it. Each check builds in the Nix sandbox. CI fails on a stale lock. A consumer borrows a lab with one line: `inputs.factory.url = "github:h0ffmann/picasso?dir=labs/factory"`.

P1 · a `labs/factory` lab: the host toolchain as one lock

A new lab exports the host tools of spec §6.1 and §9.5 as `lib.<system>.tools` and as a devShell. These tools are podman, the devcontainer CLI, go, golangci-lint, govulncheck, gitleaks, Vale, osv-scanner, d2, zensical, and Node for eslint/tsc/Playwright. **Closes:** gap 2. **Cost:** we must package oasdiff, because nixpkgs does not have it (Table 3). The `labs/lint` lab of picasso already has this shape for marola.

P2 · images from the lock: factory layer and grader

Build the factory layer (§6.1) and the grader environment (§11) with `dockerTools`, as in Experiment B. Load them into Podman. The sandbox interface of Yggdrasil does not change; only the image changes. **Closes:** gap 1. Also, the hidden-suite grader becomes the same instrument on each run. The stack layer of the realm can remain a Dockerfile in v0, because a human opens it in an editor.

P3 · an environment hash in the ledger

Add one field to the run-start event of the ledger. The field holds the hash of the `flake.lock` of the lab, or the store path of the image. Then `ygg metrics --compare` prints "environment changed" next to "factory version changed" (§10.2). **Closes:** the attribution problem of §3.4. If the first-pass rate jumps, a person can compare the jump with a toolchain bump before the journal records the jump.

P4 · fixed-output derivations for the docs toolchain

`scripts/diagrams.sh` downloads Inter and does a manual check against a pinned SHA-256. The script also pins Kroki to 0.32.1 (diagrams.md §4). The Nix `fetchurl` function with a hash does exactly this work, caches the result and fails on a mismatch. fonttools and d2 come from the same pin. The `labs/publisher` lab of picasso already builds PDFs this way, with pandoc, TeX Live and a composite GitHub Action. **Closes:** drift in rendered docs.

P5 · the canary bump as a lock diff

The resolution of ouroboric for pins versus patches (§5.2, item 4) maps one-to-one onto flake practice. The lock pins the versions, and a bump is a reviewed PR that changes one file. The CI of picasso runs with `--no-update-lock-file`, so a lock that is behind its flake causes a failure, not a warning. The canary is the first consumer that takes the bump. With Nix, the patch floor costs little to obey, because a bump is atomic and a rollback is a revert. **Closes:** the conflict, and neither side is lost.

P6 · `labs/agentic` as a second sandbox driver to measure against

The spec keeps "a plain Podman sandbox driver" as the fallback if the devcontainer spike fails (§16). Its curation also lists sandboxes that use bubblewrap. The `labs/agentic` lab of picasso packages ai-jail (bubblewrap, Landlock, seccomp) with `jail-run`. `jail-run` is a wrapper that forwards a token and has a self-test. This lab does not replace the devcontainer design, but it is a pinned, reproducible second driver that the escape-attempt suite (§13) can run against. **Closes:** nothing alone, but it gives the spike a point of comparison.

#	proposal	yggdrasil spec	effort	changes Yggdrasil?
P1	labs/factory host toolchain	§6.1, §9.5	small; oasdiff needs a package	no
P2	factory-layer and grader images from the lock	§6.1, §11	medium	image source only
P3	environment hash in the ledger	§10.1, §10.2	small	one ledger field
P4	fixed-output fetches for docs tools	diagrams.md §4	small	no
P5	canary bump as a lock diff	§9.6	process only	no
P6	agentic lab as comparison driver	§13, §16	small	no

7. Evaluation plan

Each claim above is testable. The factory already has the instruments for the tests. We propose three measurements. Each report gives the run IDs and the lock hashes that produced the measurement, in the format of the journal.

1. **Environment reproducibility.** Build the P2 images on two machines: the WSL host of the operator and a GitHub-hosted runner. Compare the digests. Success: the digests are identical, or a diffoscope report names each difference.
2. **Deterministic floor.** Run the `scripted` adapter graph ten times on the pinned environment. Success: the ledgers are identical, except for timestamps and durations. Any other difference is a nondeterminism bug in the factory. This test finds the bug before a model is involved.
3. **Stochastic ceiling.** Run the client-gallery-lite brief k times with real agents on a fixed environment. Report the spread of the hidden acceptance score, the first-pass rate and the cost per task. This spread is an estimate of the model variance, which sets the smallest improvement that the north-star measures can detect. The budget of ouroboric has caps of $5 per run and $10 per day. With these caps, $k = 5$ uses approximately three days of daily budget. The report should state that the value of k is a budget decision, not a statistical decision.

8. Limits and threats to validity

- **Nix does not make inference deterministic.** Batch-invariant kernels [24] are a property of the stack that serves the model, and a subscription user does not control that stack. The aim is to limit and measure the model variance, not to remove it.
- **Bitwise reproducibility is not universal.** In 2017–2023, 9–31% of sampled nixpkgs packages were not bitwise reproducible [5]. Rebuildability is more than 99%, and it is the stronger guarantee. A factory should record which of its images pass `--check`.
- **A pin freezes vulnerabilities.** The research of ouroboric itself gives an example: a framework pin that can ship an exploited RCE (research-web-stack §5.4). The safety of a lock is only as good as the frequency of its bumps. P5 is the countermeasure, and govulncheck and osv-scanner continue as gates.
- **Platform.** The operator works on Windows with WSL and Podman Desktop (spec §16). Nix runs inside WSL. But Nix does not fix the problem that Podman Desktop cannot see WSL paths. That problem remains a spike item.
- **Effort to learn, and ownership.** Nix is a second language next to Go. picasso is a personal repository under MIT. The factory code of ouroboric is under AGPL-3.0. An AGPL project can use MIT code. Also, the lab contract lets ouroboric copy a lab, so ouroboric does not have to depend on picasso. Thus no governance tie is necessary.
- **Our own evidence is limited.** Two rebuilds on one machine are a demonstration, not a study. The evaluation plan in §7 is necessary to make the claim of this paper true for the factory of ouroboric.

9. Conclusion

A factory that improves itself makes a claim of the form "this change caused this improvement". The literature of the last decade shows that such claims fail by default if the environment is left to chance [13, 16–20]. The supply-chain record and the CRA show that provenance is now an obligation [8–12].

Agent research shows that the model itself adds variance, and changes on the client side cannot remove this variance [23, 24, 27].

ouroboric-dev already pins two things that most projects forget: its skills and its own version. The step that remains is the layer below: the tools, the images and the grader. Functional package management makes that layer a pure function of a lock file. This is true at a measured scale of hundreds of thousands of packages [4, 5]. picasso already publishes this layer in a shape that a consumer can borrow with one line.

We recommend P1 and P3 first. They are small, and they change nothing in the design of Yggdrasil. They also make each later number in the journal traceable to the exact environment that produced it.

10. References

Each entry was checked against the record linked, on 12 October 2026.

1. E. Dolstra, M. de Jonge. Nix: A Safe and Policy-Free System for Software Deployment. *18th Large Installation System Administration Conference (LISA '04)*, USENIX, 2004. usenix.org/conference/lisa-04/nix-safe-and-policy-free-system-software-deployment
2. E. Dolstra, A. Löh, N. Pierron. NixOS: A purely functional Linux distribution. *Journal of Functional Programming* 20(5–6):577–615, 2010. doi:10.1017/S0956796810000195
3. L. Courtès, R. Wurmus. Reproducible and User-Controlled Software Environments in HPC with Guix. *Euro-Par 2015 Workshops*, LNCS, 579–591, 2015. doi:10.1007/978-3-319-27308-2_47
4. J. Malka, S. Zacchiroli, T. Zimmermann. Reproducibility of Build Environments through Space and Time. arXiv:2402.00424, 2024.
5. J. Malka, S. Zacchiroli, T. Zimmermann. Does Functional Package Management Enable Reproducible Builds at Scale? Yes. arXiv:2501.15919, 2025.
6. C. Lamb, S. Zacchiroli. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39(2):62–70, 2022. doi:10.1109/MS.2021.3073045
7. K. Thompson. Reflections on trusting trust. *Communications of the ACM* 27(8):761–763, 1984. doi:10.1145/358198.358210
8. M. Ohm, H. Plate, A. Sykosch, M. Meier. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. *DIMVA 2020*, LNCS, 23–43, 2020. doi:10.1007/978-3-030-52683-2_2
9. P. Ladisa, H. Plate, M. Martinez, O. Barais. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. *2023 IEEE Symposium on Security and Privacy*, 1509–1526, 2023. doi:10.1109/SP46215.2023.10179304
10. NIST National Vulnerability Database. CVE-2024-3094 (xz-utils), published 29 March 2024. nvd.nist.gov/vuln/detail/CVE-2024-3094
11. OpenSSF. SLSA specification, version 1.2, build track levels. slsa.dev/spec/v1.2
12. Regulation (EU) 2024/2847 of the European Parliament and of the Council on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act). *Official Journal of the European Union*, L series, 2024. eur-lex.europa.eu/eli/reg/2024/2847/oj
13. M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature* 533(7604):452–454, 2016. doi:10.1038/533452a
14. R. D. Peng. Reproducible Research in Computational Science. *Science* 334(6060):1226–1227, 2011. doi:10.1126/science.1213847

15. National Academies of Sciences, Engineering, and Medicine. *Reproducibility and Replicability in Science*. The National Academies Press, 2019. doi:10.17226/25303
16. C. Collberg, T. A. Proebsting. Repeatability in computer systems research. *Communications of the ACM* 59(3):62–69, 2016. doi:10.1145/2812803
17. A. Trisovic, M. K. Lau, T. Pasquier, M. Crosas. A large-scale study on research code quality and execution. *Scientific Data* 9:60, 2022. doi:10.1038/s41597-022-01143-6
18. J. F. Pimentel, L. Murta, V. Braganholo, J. Freire. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. *MSR 2019*, 507–517, 2019. doi:10.1109/MSR.2019.00077
19. O. E. Gundersen, S. Kjensmo. State of the Art: Reproducibility in Artificial Intelligence. *AAAI 2018*, 32(1), 2018. doi:10.1609/aaai.v32i1.11503
20. K. Hinsen. Dealing With Software Collapse. *Computing in Science & Engineering* 21(3):104–108, 2019. doi:10.1109/MCSE.2019.2900945
21. M. D. Wilkinson et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3:160018, 2016. doi:10.1038/sdata.2016.18
22. C. Boettiger. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* 49(1):71–79, 2015. doi:10.1145/2723872.2723882
23. S. Ouyang, J. M. Zhang, M. Harman, M. Wang. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. arXiv:2308.02828, 2023.
24. H. He and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab blog, 10 September 2025. thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference
25. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *AISec '23*, 79–90, 2023. doi:10.1145/3605764.3623985
26. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023.
27. Y. Zhu et al. Establishing Best Practices for Building Rigorous Agentic Benchmarks. arXiv:2507.02825, 2025.
28. Reproducible Builds project. Definitions. reproducible-builds.org/docs/definition
29. ouroboric-dev. Repositories ouroboric (4ab80ed), yggdrasil (54ea51f), asgard (8bdbc03), .github (b799793), CC BY 4.0 documentation. github.com/ouroboric-dev
30. h0ffmann. picasso: Nix labs (AGENTS.md, PHILOSOPHY.md, labs/). github.com/h0ffmann/picasso